

В сортировке подсчётом предполагается, что все n входных э-ов — целые числа, принадлежащие интервалу от 0 до k , где k — некоторая целая константа. Если $k = O(n)$, то время работы алгоритма $\Theta(n)$.

Основная идея сортировки подсчётом заключается в том, чтобы для каждого входного элемента x определить кол-во элементов, которые меньше x . С помощью этой информации можно получить позицию элемента x , где он должен находиться в отсорт. последовательности.

При составлении кода предполагается, что на вход подаётся массив $A[1 \dots n]$, так, что $\text{len}(A) = n$.
Потребуются ещё два массива:
- Массив $B[1 \dots n]$ с отсортированной входной послед-стью.
- Массив $C[1 \dots k]$ служит временным рабочим хранилищем.

CountingSort(A, B, k)

1 for $i = 0$ to k :

2 $C[i] = 0$

3 for $i = 1$ to $\text{length}(A)$:

4 $C[A[i]]++$

5 // в $C[i]$ хранится число э-ов, равных i

6 for $i = 1$ to k :

Сколько же по времени?

1-2 необходимо $\Theta(k)$

3-4 необходимо $\Theta(n)$

6-7 необходимо $\Theta(k)$

```

7   C[i] += C[i-1]
8 // В C[i] кол-во э-ов, не треб. i
9 for i = length(A) to 1:
10    B[C[A[i]]] = A[i]
11    C[A[i]] --

```

9-11 необходимо $\Theta(n)$

По скольку значения могут повторяться необходимо выпавить строку 11, благодаря этому след. эл-т равный $A[i]$ будет помещаться перед ним.

Такое время работы алгоритма $\Theta(n+k)$, на практике алгоритм применяется когда макс. значение ключа не превышает кол-во элементов, $\Rightarrow \Theta(n)$.

Алгоритм сортировки подсчётом является стабильным. Т.к. в строках 9-11 мы проходим справа-налево.